CS780 Capstone Project:
OBELIX: the Warehouse Robot

FirstName <Middle Name if any> LastName
roll no.
arpity22@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

OBELIX is a partially observable warehouse-robot task where the agent must locate a box using sparse binary sensors, attach to it, and push it to the boundary while avoiding repeated "stuck" events and wall obstacles. We implemented a progression of reinforcement learning agents: Double/Dueling DQN with prioritized experience replay, recurrent Q-learning variants to handle blinking/moving targets, and several PPO stabilizations. Our final submission focuses on a recurrent actor-critic trained with PPO plus auxiliary intrinsic learning (random network distillation) and deterministic search/stuck-recovery heuristics for robustness. We summarize training dynamics and local evaluation evidence across the three difficulty levels.

1 Competition Result

Codabench Username: a_220206
Final leaderboard rank (Test Phase): 45
Leaderboard rank – Level 1 (static): 36
Leaderboard rank – Level 2 (blinking): 71
Leaderboard rank – Level 3 (moving + blinking): 70

The repository contains 29 locally packaged "submission" directories used during development (many also contain a ready-to-upload `submission.zip`). Table 1 summarizes them by method family.

Table 1: Local submission packages created during development (repository evidence).

Method family	Count	Examples (directories)
D3QN-PER (dueling+double+PER)	11	`submission_001`, `_002_no_wall`, `phase4_final`
PPO variants	7	`submission_ppo_test`, `_adaptive_v1`, `_adv_final`
RAPTOR-PPO variants	5	`submission_raptor_ppo` (& `_ppo1--_ppo4`)
Recurrent DQN variants (ADRQN/RD3QN)	3	`submission_adrqn.py`, `submission_rd3qn`
World model (POMDP)	1	`submission_worldmodel_pomdp1`
Debug/other	2	`submission_debug_tmp`, `submission_improved_lstm`

2 Introduction and Problem Description

OBELIX is a simulated warehouse-robot domain inspired by the classic behavior-based robotics setting by Mahadevan et. al [1]. The agent controls a differential-drive robot using a small discrete action set: turn left/right by 22.5° or 45°, or move forward. At each step the robot receives an 18-dimensional binary observation summarizing local proximity sensing (sonar bits), an IR cue, and a "stuck" "stuck" flag. The goal is to locate the grey box, attach to it, and then push the attached box until it touches the arena boundary, which terminates the episode with a large terminal reward.

The environment exposes three difficulty modes. Level 1 (difficulty 0 in code) uses a static box. Level 2 (difficulty 2) makes the box intermittently visible (blinking), which creates partial observability because the same local sensor readings can correspond to multiple latent box positions. Level 3 (difficulty 3) additionally moves the box while blinking, introducing non-stationarity and requiring both memory and robust interception behavior. An additional robustness axis enables wall obstacles, which creates dead-ends and increases the probability of repetitive collision loops.

Overall, this is a POMDP: the current observation is insufficient to fully infer the box pose, the robot pose relative to obstacles, or whether the box has recently blinked out of view. The high-level objective in this project was to maximize mean cumulative reward under the Codabench evaluation protocol (multiple random seeds, fixed horizon) across all three difficulties while remaining robust to wall obstacles.

3 Approach and Model Evolution

We followed an iterative "baseline → stabilize → add memory → improve exploration" workflow. The main bottlenecks observed early were (i) sparse successful trajectories (attach+push to boundary), (ii) severe penalties from the stuck flag, and (iii) partial observability at higher levels (blinking and moving targets). This led to two broad families of solutions: value-based replay methods that can reuse rare successes, and on-policy actor-critic methods that can incorporate recurrent memory and entropy-based exploration.

3.1 Value-based baselines: Double/Dueling DQN with PER

Our initial agents used Double DQN [2] to reduce overestimation, and then moved to a dueling architecture [3] with prioritized experience replay (PER) [4]. The dueling split helps learn state value even when actions are not immediately differentiating, while PER increases the sampling rate of high TD-error transitions (often rare near-success or recovery events). For evaluation, we used greedy action selection with a small "action smoothing" rule to reduce oscillations when the top Q-values are nearly tied.

These agents operate directly on the raw 18-bit observation vector, which is attractive for simplicity but limits the ability to infer latent box motion or handle blinking without history.

3.2 POMDP extensions: recurrent value learning and auxiliary belief

To address blinking/moving targets, we explored recurrent value learning (ADRQN/RD3QN-style) by augmenting the Q-network with an LSTM/GRU over recent observations and actions. In addition, one variant introduced a simple spatial "belief" map feature to stabilize navigation under partial observability. The recurrent models improved qualitative tracking in Level 2/3, but were sensitive to hidden-state reset behavior and to train–test mismatches in how episode boundaries are detected.

3.3 On-policy actor-critic: PPO variants and stabilizations

We implemented PPO [5] with generalized advantage estimation (GAE) [6] and iterated through several stabilizations: timeout-aware bootstrapping, entropy schedules to balance exploration/exploitation, and optional recurrent policies. PPO was especially convenient for experimenting with reward-shaping knobs (e.g., encouraging forward motion when front sensors are active) and for controlling exploration through an entropy bonus.

3.4 Final model: recurrent PPO with intrinsic learning (RAPTOR-PPO)

Our final submission is a recurrent actor-critic trained with PPO and augmented with intrinsic motivation via random network distillation (RND) [7] and an auxiliary next-observation prediction head. The motivation is that Level 2/3 require persistent exploration under intermittent observability, and intrinsic novelty rewards help discover attach/push behaviors more reliably.

Architecture and state representation. The policy consumes an augmented feature vector built from the raw observation $o_t \in \mathbb{R}^{18}$, a first-difference feature $\Delta o_t = o_t - o_{t-1}$, a one-hot encoding of the previous action, and two small scalars that summarize "time since last seeing the target" and normalized episode progress (43 dimensions total). This is passed through an MLP encoder and a GRU memory. The GRU hidden state feeds a policy head (5-way logits) and a value head. Figure 1 shows a schematic.

Raw sensors o_t (18D) $\rightarrow$ feature augmentation (delta, last-action one-hot, timers) $\rightarrow$ MLP encoder $\rightarrow$ GRU memory $\rightarrow$ {policy logits, value}.

Figure 1: Schematic of the final recurrent actor-critic (RAPTOR-PPO) used for evaluation.

Exploration and shaping. PPO exploration is controlled primarily through an entropy bonus. For the intrinsic component, an RND predictor provides a novelty signal that is annealed down over training so that the final policy relies on extrinsic reward. We also included deterministic "search" and "stuck recovery" heuristics in the action selection logic (e.g., turn–turn–forward cycles when the stuck flag is raised) to increase robustness with walls; this mirrors the behavior-based decomposition spirit of [1] even though the final policy is trained end-to-end.

4 Experiments and Implementation Details

All training and evaluation code is implemented in Python. The OBELIX simulator is in `obelix.py` (OpenCV-based dynamics and sensing). For reproducibility we used fixed evaluation settings consistent with the provided evaluator: horizon 1000 steps, wall obstacles enabled for robustness tests, and difficulty levels $\{0, 2, 3\}$. During training, we checkpointed periodically and selected submission checkpoints based on deterministic evaluation runs.

Most experiments were run CPU-only (matching the Codabench inference constraint). Training budgets varied by variant: value-based runs typically used 1000–4000 episodes with 1000 max steps, while PPO variants used 1–2 million environment steps with parallel rollout workers. Based on the stored training logs, end-to-end wall-obstacle runs ranged from roughly $\sim$33 minutes (e.g., `submission_phase4_final`) to $\sim$158 minutes (e.g., `submission_001`) depending on algorithm and data-collection parallelism.

We also explored curriculum-style training (`train_curriculum.py`) that progressively increased difficulty and/or reduced shaping as performance improved. While this improved early learning stability, the final strategy prioritized robustness on the hardest settings (Level 3 with walls) via recurrent policies and explicit stuck-recovery behavior.

Table 2 summarizes representative hyperparameters for the major agent families. For value-based methods, we used an Adam optimizer, replay buffers up to 10^5 transitions, and periodic target-network synchronization. For PPO, we used GAE, mini-batch SGD over rollout data, and (in the recurrent variants) truncated backpropagation through time over short sequences.

Table 2: Representative hyperparameters used for major model variants (defaults as implemented in the training scripts).

Variant	γ	LR	Batch/rollout	Notable settings
D3QN-PER [4, 3, 2]	0.99	5×10^{-4}	256	replay 10^5, PER $\alpha = 0.6$, $\beta : 0.4 \to 1$
PPO baseline [5]	0.99	3×10^{-4}	2048/256	10 epochs, clip 0.2, entropy bonus ≈ 10
RAPTOR-PPO (final)	0.995	3×10^{-4}	512 horizon	GRU(192), RND intrinsic coef annealed
World-model (exploratory) [8]	0.995	$\sim 2 \times 10^{-4}$	seq 32 / 64	latent model + imagination rollouts (not

Local evaluation. We used `evaluate.py` to compute mean and standard deviation of episodic scores over multiple random seeds, appending results to `leaderboard.csv`. To approximate the Codabench protocol, we also used `evaluate_on_codabench.py`, which runs 10 episodes per difficulty with wall obstacles enabled and reports mean/std scores per level.

5 Results

This repository contains rich development evidence (training logs, diagnostic plots, and local evaluation summaries). However, official Codabench ranks are not stored in the repository, so the "Test Phase" leaderboard fields in Section 1 must be filled from the Codabench page.

5.1 Local leaderboard snapshots

The file `leaderboard.csv` stores local evaluation rows (mean/std scores over multiple runs). An example snapshot for the provided template agent shows strongly negative mean scores at high difficulty with walls, indicating that avoiding repeated stuck penalties is critical.

5.2 Training-log evidence across phases

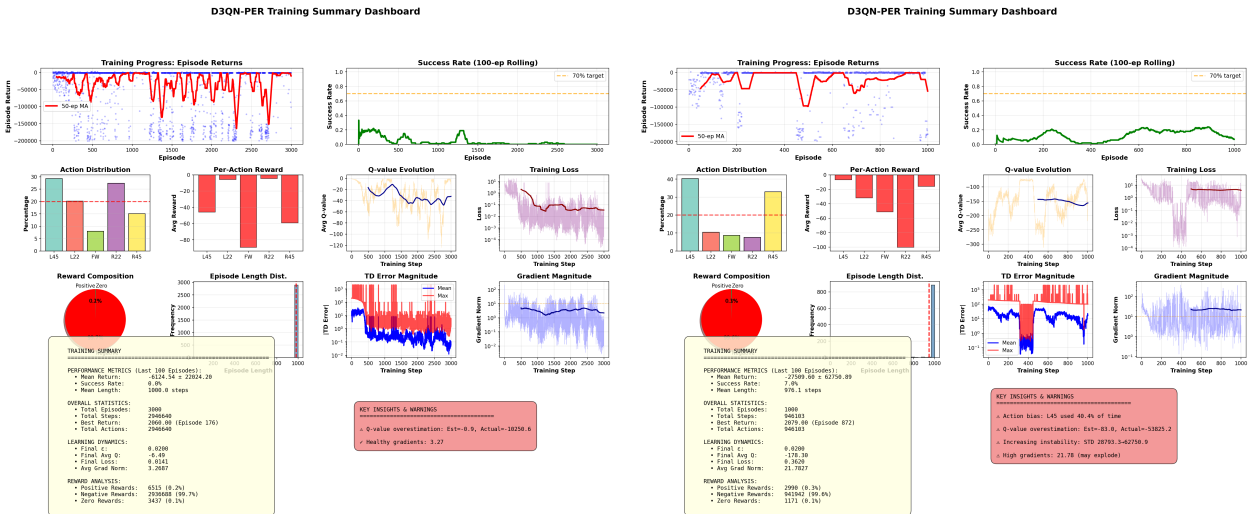
Several development submissions include `training_log.txt` with aggregate returns and success rates. Table 3 highlights representative runs.

Table 3: Representative training-log summaries from development submissions (success = fraction of episodes reaching the terminal attach+boundary condition as logged by the trainer).

Submission	Episodes	Mean return	Success rate	Best success
`submission_001`	3000	-108260	16%	32%
`submission_002_no_wall`	2000	-1185	0%	24%
`submission_FIXED_shaped`	4000	-30453	7%	30%
`submission_phase2_medium`	2000	-116957	16%	17%
`submission_phase4_final`	1000	-27510	7%	23%

5.3 Learning curves and diagnostics

Figure 2 shows two representative training summaries from D3QN-PER development runs. The plots show high variance in episodic returns and sparse success events, consistent with a task dominated by step penalties and occasional large terminal rewards.



(a) D3QN-PER (no wall obstacles).

(b) D3QN-PER (final shaped phase).

Figure 2: Representative learning curves from development submissions (generated by training scripts).

Which model performed best and why? In our development workflow, the strongest improvements came from (i) stabilizing learning dynamics (target networks / PPO clipping), (ii) adding memory for blinking/moving targets (recurrent policies), and (iii) enforcing robust recovery behavior when the stuck flag becomes active (crucial with wall obstacles). Intrinsic motivation helped exploration early, but required annealing so that the final behavior optimizes extrinsic success.

6 Error Analysis and Discussion

Failure modes by difficulty. Even in Level 1, failures occur when the robot enters local limit cycles: repeated turning in place, "bouncing" off walls, or approaching the box at an angle that causes immediate stuck penalties. In Level 2, blinking introduces false negatives: the agent can

lose track of the box after it disappears, making greedy policies overreact to short-term sensor noise. Level 3 adds box motion, which creates interception errors and increases the likelihood of overshooting or pushing the box into awkward corners.

Walls and stuck dynamics. With wall obstacles enabled, the dominant failure mode is getting trapped in corners or narrow passages and repeatedly triggering the stuck flag (large negative penalties). Robust policies require an explicit recovery behavior (e.g., alternating turns and forward motion) and should avoid committing to tight turns when front sensors indicate imminent collision.

Evaluation mismatches. One subtle issue we encountered is that some training/evaluation utilities infer "success" using a reward threshold, while the environment applies per-step penalties even in successful episodes. This can underreport success in logs and can mis-rank checkpoints. A more reliable definition is to derive success from the terminal condition (attached box reaching the boundary) rather than from reward magnitude.

What we would improve with more time. Two clear directions are (i) making POMDP handling more principled (e.g., better recurrent state management or latent-state models [8]), and (ii) improving wall-robust navigation by explicitly learning a short-horizon obstacle-avoidance controller that can be composed with the box-seeking/pushing behavior.

7 Conclusion

We explored a broad design space for OBELIX, progressing from replay-based value learning to recurrent actor-critic methods with intrinsic motivation. The main lesson is that partial observability and "stuck" dynamics dominate performance: memory helps track intermittent targets, but robustness ultimately depends on reliable recovery behaviors and evaluation discipline (consistent success definitions and checkpoint selection). Future work could incorporate stronger latent-state world models, richer observations, and more realistic physics to reduce the gap between simulation strategies and real-robot robustness.

LLM Usage Declaration

- **Which LLM was used?** GitHub Copilot Chat (GPT-5.2) was used to help structure the report and to summarize code/experiment artifacts in the repository.
- **In which part was the LLM used, and how did it help?**

Part	Used LLM? (Yes/No)	Purpose of Using LLM
Understanding concepts	Yes	Clarifying PPO/DQN variants and POMDP fra
Ideation / brainstorming	Yes	Organizing the approach-evolution narrative
Debugging small code snippets	Yes	Minor checks and small refactors during develo
Plotting / visualization code	No	(Plots were generated by existing training scrip
Grammar / writing style	Yes	Editing for clarity and concision
Report structure / section ideas	Yes	Mapping repository evidence into the template
Other (specify)	No	

- **Other sources used (besides the LLM and slides).** We referenced standard RL papers for DQN/PPO/PER/RND and the original OBELIX paper [1]. We also consulted OpenCV documentation as needed for environment rendering/debugging.
- **Own contribution.** We implemented multiple agents and training pipelines in PyTorch, designed and tuned reward shaping/exploration schedules, ran systematic experiments across difficulty levels (including wall robustness), and performed checkpoint selection and error analysis based on local evaluator logs and diagnostic plots.

References

[1] S. Mahadevan and J. H. Connell, "Automatic programming of behaviour-based robots using reinforcement learning," in *Proceedings of the Ninth National Conference on Artificial Intelligence (AAAI-91)*, 1991.

[2] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, 2016.

[3] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," in *International Conference on Machine Learning (ICML)*, 2016.

[4] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *International Conference on Learning Representations (ICLR)*, 2016.

[5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[6] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *arXiv preprint arXiv:1506.02438*, 2016.

[7] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, "Exploration by random network distillation," in *International Conference on Learning Representations (ICLR)*, 2019.

[8] D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba, "Dream to control: Learning behaviors by latent imagination," in *International Conference on Learning Representations (ICLR)*, 2020.